TECHNICAL TRAINING

Distributed System Design

Reasoning about failure, consistency and boundaries in systems that cannot be made to behave like a single machine.

Vivekanand Pujar | @VivekanandPujar

Consultant-led workshop · Senior engineers, architects and engineering leadership

Seven sessions · Trade-offs, anti-patterns and current practice

SESSION MAP

Agenda

01 **How distributed systems behave differently**

Partial failure, no shared clock and unbounded latency change what a correct program is.

02 **The fallacies of distributed computing**

Eight assumptions inherited from single-process code, and the designs each one produces.

03 **CAP and the problems of distributed data**

What the theorem actually proves, and the data problems it does not address.

04 **CAP and PACELC in the real world**

Why the latency branch governs daily design, and where real engines sit.

05 **Essential domain-driven design**

Bounded contexts and aggregates as the seams along which a system can be cut.

06 **The pillars of distributed system design**

Partitioning, replication, coordination, communication, resilience, observability.

07 **DevOps and infrastructure as code**

Why a design the team cannot deploy safely is not a design at all.

THE ARGUMENT

What this course is trying to establish

> A distributed system will not sell you correctness, availability and latency together. Competent design is choosing the currency deliberately, operation by operation, and building the machinery to survive the choice.

Choose per operation, not per system

A checkout and a product listing do not need the same guarantee. A single consistency model applied everywhere overpays on the read path and under-protects the money path.

The boundary decides the cost

Most consistency pain is a modelling failure: one invariant was split across two services. Redraw the boundary and the distributed transaction disappears rather than being solved.

A design is not finished until it is operable

A correct topology that cannot be deployed, observed or reversed independently behaves in production exactly like an incorrect one.

SESSION 01

A distributed system fails in ways a single process cannot express

Partial failure

A call may fail after the work has already been performed. the caller cannot distinguish a lost request, a lost response and a peer that is merely slow.

BREAKS THE ASSUMPTION

That a function either runs or does not run.

SHOWS UP AS

Duplicate orders, repeated refunds, and workflows waiting on a reply that was already delivered.

No global clock

Wall clocks drift and step. Ordering events by timestamp silently discards writes, so causality has to be carried explicitly in the data.

BREAKS THE ASSUMPTION

That all participants share one notion of now and after.

SHOWS UP AS

Last-write-wins dropping the surviving update; leases expiring early; logs that cannot be interleaved.

Unbounded latency

The network offers no upper bound on delivery. Every timeout is a guess about a value the system will never reveal.

BREAKS THE ASSUMPTION

That a call completes within a bounded time.

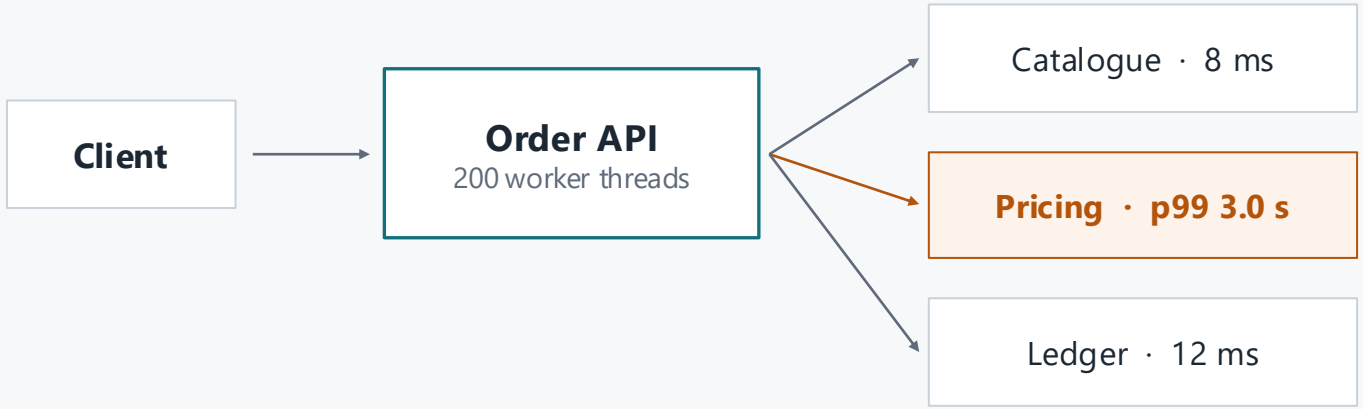
SHOWS UP AS

Queues growing faster than they drain, and one slow dependency exhausting a shared thread pool.

Consequence: a correctness argument must hold under arbitrary loss, reordering and delay — not merely under the failures the team has already witnessed.

SESSION 01

One slow dependency takes down the services that are healthy



ORDER API WORKER POOL



At 60 requests per second, a dependency with a three-second p99 occupies 180 of the 200 workers. Catalogue and Ledger are healthy, and unreachable.

Hystrix popularised bulkheads and breakers in 2012; the mechanism now lives in Resilience4j, Polly, Envoy outlier detection and service-mesh policy.

THE INTUITIVE FIXES

Add more threads.

The queue moves rather than disappears; memory and scheduling become the next limit.

Retry the slow call.

Load is multiplied against the dependency that is already failing.

Raise the timeout.

A fast failure becomes a slow outage, and it propagates upstream.

WHAT ACTUALLY HOLDS

- Bounded, separate pool per dependency.
- Timeout set from the caller's remaining budget.
- Circuit breaker with half-open probing.
- Shed load at the edge; degrade to a cached price.

Vivekanand Pujar | @VivekanandPujar

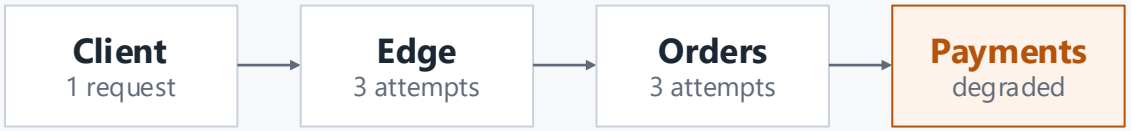
SESSION 02

The eight fallacies are not myths; each one names a design that ships

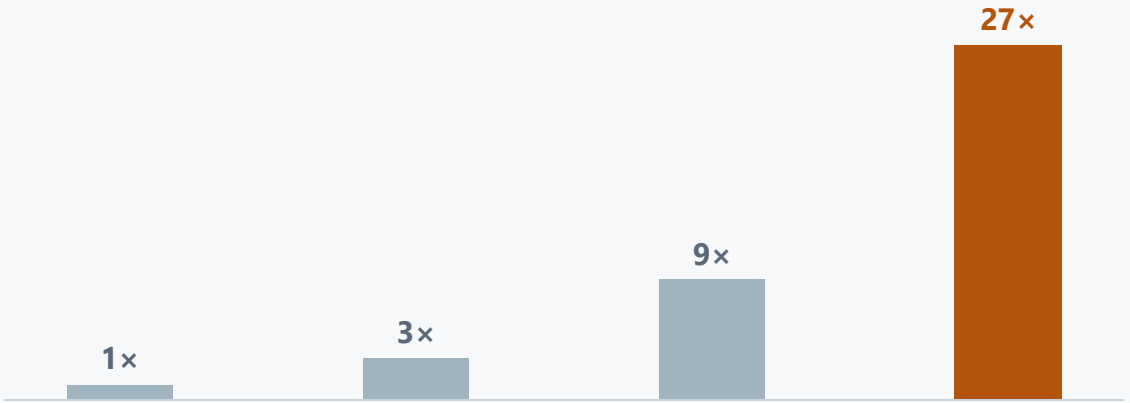
THE ASSUMPTION	THE DESIGN IT PRODUCES	WHAT BREAKS IN PRODUCTION
The network is reliable	Fire-and-forget calls; no idempotency key; no outbox	Silent message loss and duplicate side effects on retry
Latency is zero	Chatty request-per-item loops across a service boundary	A page that is fast in staging and times out at scale
Bandwidth is infinite	Whole aggregates returned where a projection would do	Egress cost, serialisation load and cache thrash
The network is secure	Trust granted by network position; unauthenticated internal calls	One breached pod reaching every service; no audit trail
Topology does not change	Hard-coded hosts, pinned instances, sticky sessions	Autoscaling and failover break the assumptions of the caller
There is one administrator	Shared database owned by nobody; coordinated releases	Change velocity falls to that of the slowest team
Transport cost is zero	Synchronous chains treated as free composition	p99 that is the sum of every hop, plus the tail of each
The network is homogeneous	One timeout, one retry policy, one serialisation format	Cross-region and cross-cloud paths behave nothing like local ones

SESSION 02

The retry is what turns a degradation into an outage



LOAD ARRIVING AT EACH TIER, RELATIVE TO CLIENT DEMAND



Three tiers retrying three times each present twenty-seven requests to the tier that was already failing. The amplification is multiplicative, and it peaks precisely when capacity is lowest.

WHY THE INSTINCT IS WRONG

A retry is positive feedback. Under overload it adds exactly the load that produced the failure.

Retries at every layer multiply rather than add; the caller cannot see the total it is generating.

Unjittered backoff synchronises clients, so the herd returns together and the recovery fails.

PRACTICE

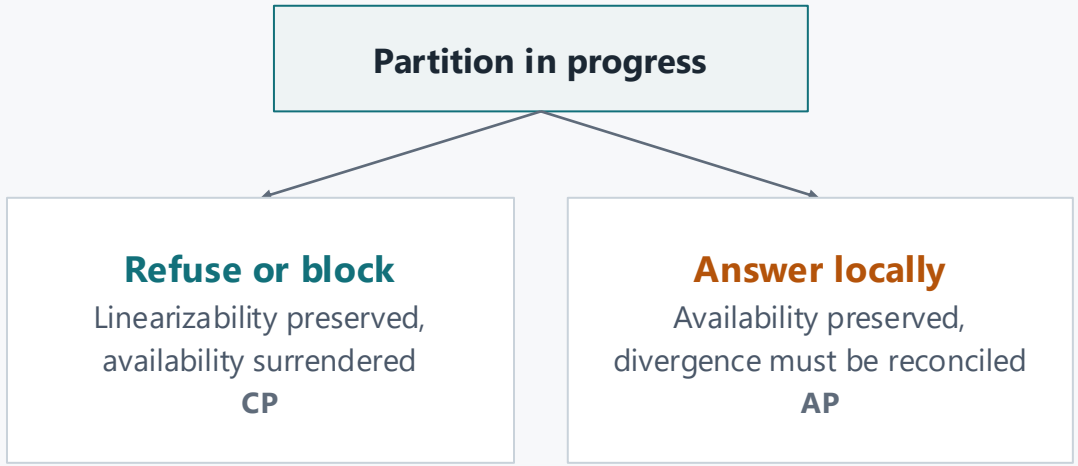
- Retry at one layer only, and record which.
- Cap retries as a share of live traffic.
- Exponential backoff with full jitter.
- Retry only idempotent calls, carrying a key.
- Propagate the deadline with the request.

SESSION 03

CAP constrains one narrow situation, and is usually quoted outside it

WHAT THE THEOREM ACTUALLY STATES

While a network partition is in progress, a replicated store cannot serve every request and remain linearizable. One of the two must be surrendered.



Only this branch is constrained. Every other guarantee in the architecture is a separate argument that CAP does not settle.

THREE MISREADINGS

"Pick any two of the three."

The trade arises only during a partition. Absent one, a system may offer both. CA is not a choice; it describes a system that has given up on partitions, which the network will not permit.

"C means the data is correct."

C is linearizability, a real-time ordering guarantee over single objects. The C in ACID is invariant preservation. The two share a word and nothing else.

"A means high uptime."

A requires every non-failing node to answer every request. A cluster that returns 503 during a partition has excellent uptime and no CAP availability at all.

Brewer, PODC 2000 (conjecture); Gilbert and Lynch, SIGACT News 2002 (proof); Brewer, "CAP Twelve Years Later", IEEE Computer, 2012.

SESSION 04

Vivekanand Pujar | @VivekanandPujar

Partitions are rare; latency is charged on every request

if (**P**) then (**A** or **C**) else (**L** or **C**)

P partition A availability C consistency L latency

THE BRANCH THAT RUNS EVERY DAY

A partition is an incident measured in minutes per year. The latency-versus-consistency choice is paid on every read and every write.

A quorum acknowledgement across availability zones costs a round trip; across regions it costs tens of milliseconds, on the hot path, permanently.

Most architecture arguments that invoke CAP are in fact arguments about the E branch: where reads are served from, and how far a write must travel before it is acknowledged.

INDICATIVE ROUND TRIP FOR A QUORUM

Within one availability zone	**0.3 – 0.7 ms**
Across zones, one region	**1 – 2 ms**
North Virginia to Ireland	**70 – 90 ms**

Strong consistency within a region is usually affordable. Strong consistency across regions is a product decision rather than a database setting, because it appears in every user-visible latency budget.

PACELC: Abadi, "Consistency Trade-offs in Modern Distributed Database System Design", IEEE Computer, 2012. Latency figures are indicative public-cloud observations.

Vivekanand Pujar | @VivekanandPujar

Where the engines you actually use have placed the bet

SYSTEM	DEFAULT	WHAT THE DEFAULT BUYS	WHAT IT COSTS
Cloud Spanner	**PC/EC**	External consistency across regions, underwritten by TrueTime	Commit wait on every write; topology dictates the latency floor
CockroachDB	**PC/EC**	Serializable transactions on commodity clocks, with geo-partitioning	A misplaced leaseholder quietly becomes the p99 of the service
MongoDB (majority)	**PC/EC**	Primary reads and majority writes yield read-your-writes by default	Failover stalls writes; relaxing to w:1 reintroduces silent loss
DynamoDB	**PA/EL**	Predictable low latency, and service during host and zone failure	Reads are eventually consistent unless paid for, and then single-region
Cassandra, ScyllaDB	**PA/EL**	Per-query quorum tuning; multi-datacentre writes remain available	Reconciliation becomes application logic; last-write-wins discards data
Kafka (acks=all)	**PC/EC**	An ordered, replicated commit point per partition	Producer latency tracks the slowest in-sync replica
etcd, ZooKeeper, Consul	**PC/EC**	Linearizable metadata and leader election for everything else	The minority side is unavailable by design; not a general database

Every posture is a default and is configurable per operation: the engine sets the default, the design sets the exception.

SESSION 03

Consistency is a dial with named settings, not a switch

Few applications require linearizability everywhere. The engineering question is the weakest model that still preserves the invariant in question.

STRONGER GUARANTEE → LOWER COORDINATION COST

Linearizable

Every read observes the most recent committed write, in real time.

A round trip, per operation

Sequential

All participants observe one order; it need not match real time.

Ordering without the clock

Causal

Operations that depend on one another are observed in that order.

Metadata carried per key

Session

A client reads its own writes and never moves backwards.

Sticky routing of reads

Eventual

Replicas converge once writes cease, with no ordering promise.

Reconciliation becomes yours

THE INTUITIVE DEFAULT

Selecting the strongest mode the engine offers. It converts a latency budget into a coordination budget, and conceals which operations genuinely required it.

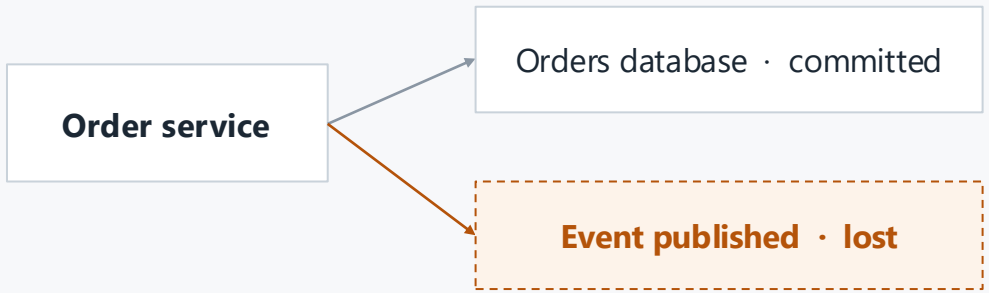
THE PRACTICE

State the invariant, select the weakest model that preserves it, and record that decision beside the endpoint so the next engineer inherits the reasoning.

SESSION 03

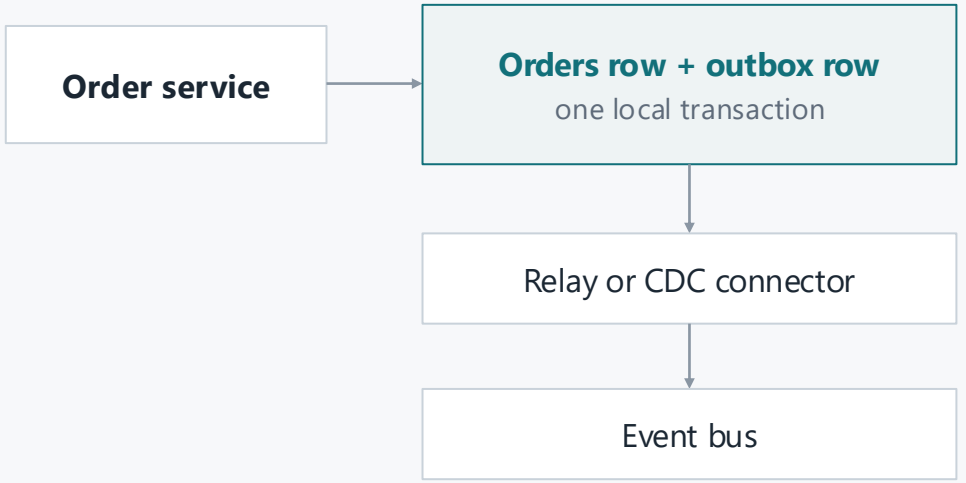
The dual write is the most common way correctness is quietly lost

THE PATTERN THAT LOOKS OBVIOUS



No atomic step spans two systems. A crash, a timeout or a rollback between the two writes leaves the database and the log permanently disagreeing, and nothing detects it.

THE PATTERN THAT HOLDS



One transaction, then asynchronous delivery. Delivery is at-least-once, so every consumer must be idempotent.

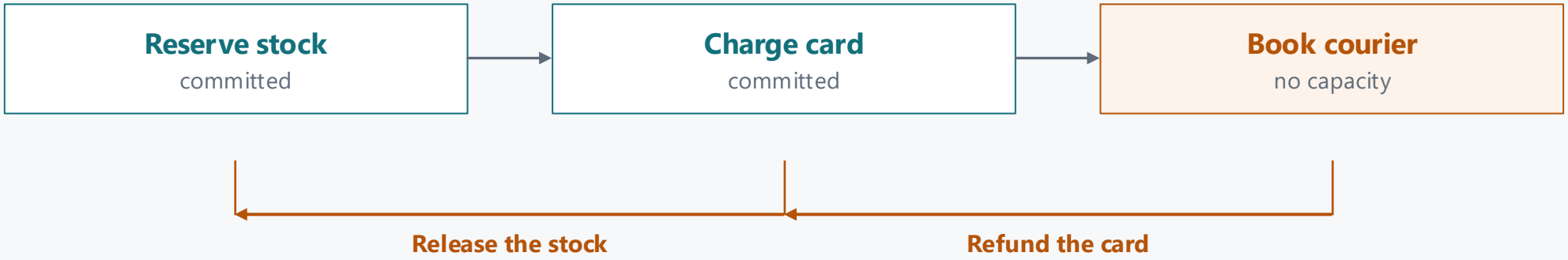
Historically: XA and two-phase commit did provide atomicity across resources, at the price of a blocking coordinator. A coordinator that fails holds locks until it returns, which is why the pattern left internet-scale designs.

Current practice: transactional outbox with Debezium or native change data capture, Kafka transactions, DynamoDB Streams, PostgreSQL logical replication, Azure Service Bus sessions.

Vivekanand Pujar | @VivekanandPujar

Once the transaction is gone, the undo becomes part of the design

A THREE-STEP SAGA, FAILING AT THE LAST STEP



Compensating actions run in reverse order. Each is a new business fact, not the erasure of an old one.

Orchestration or choreography

An orchestrator makes the process explicit, testable and observable, at the cost of centralising domain knowledge. Choreography keeps services independent, but the process then exists only in the wiring.

Compensation is not rollback

A refund is a new fact; the charge remains in the ledger. Intermediate states are therefore visible to customers and to auditors, so the domain model must name them honestly: reserved, pending, reversed.

The standing obligations

Every step idempotent and retryable. A saga log that survives process restart. A deadline on each step. And a defined, alerted outcome for the case in which compensation itself fails.

SESSION 05

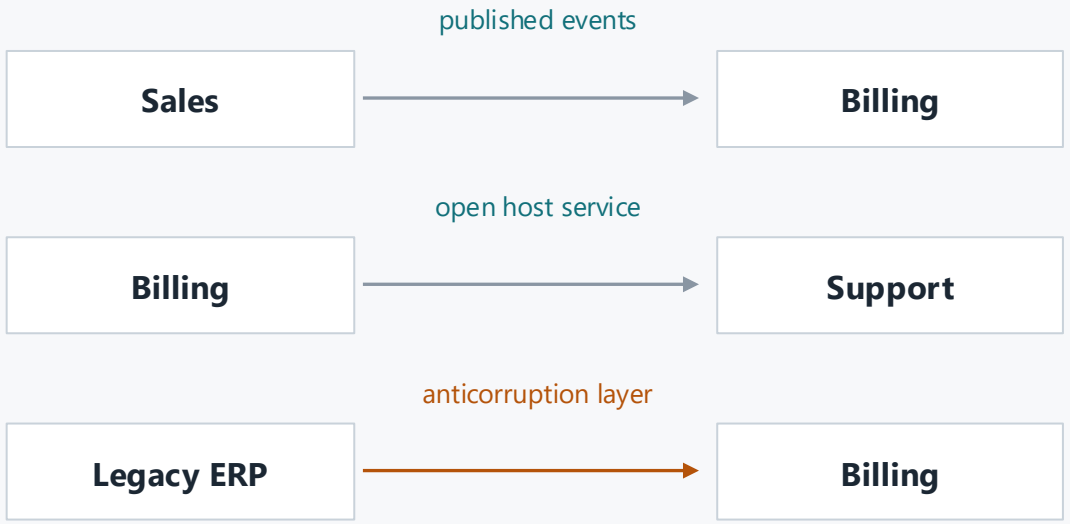
A bounded context is the seam along which a system can be cut

ONE WORD, THREE MEANINGS

Sales	Customer is a lead: a pipeline stage, a probability and an owner.
Billing	Customer is a legal entity: a tax identity, an address and a payment instrument.
Support	Customer is a person: a handle, an entitlement and a history of contact.

A single shared Customer table obliges three teams to agree on one definition that serves none of them, and couples their release schedules permanently.

THE CONTEXT MAP IS THE ARCHITECTURE



The map records who depends on whom, and on what terms. An anticorruption layer is the price of importing a model you do not control.

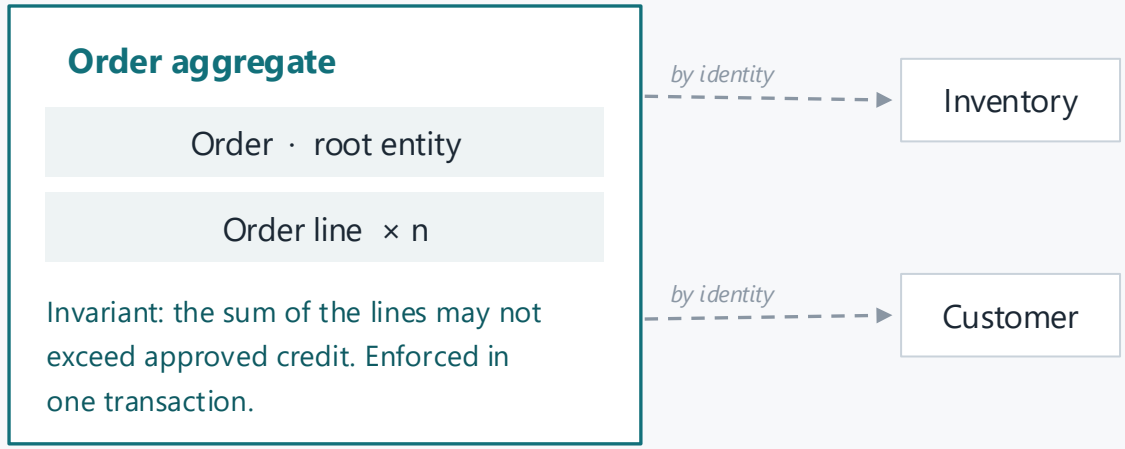
The test in practice: if two teams must change together to ship one feature, the boundary is drawn in the wrong place.

Vivekanand Pujar | @VivekanandPujar

SESSION 05

The aggregate is the one place where strong consistency stays cheap

ONE TRANSACTION, ONE AGGREGATE



Everything beyond the boundary is referenced by identity and brought into step through domain events. It is eventually consistent by construction, and says so.

DRAWN TOO LARGE

Lock contention and long transactions. The aggregate becomes the service, and unrelated writes queue behind one another. The symptom is a hot row and a rising rate of write conflicts.

DRAWN TOO SMALL

Invariants leak outward into application services and sagas. The symptom is one business rule enforced in three handlers, owned by nobody, and correct in two of them.

THE WORKING RULE

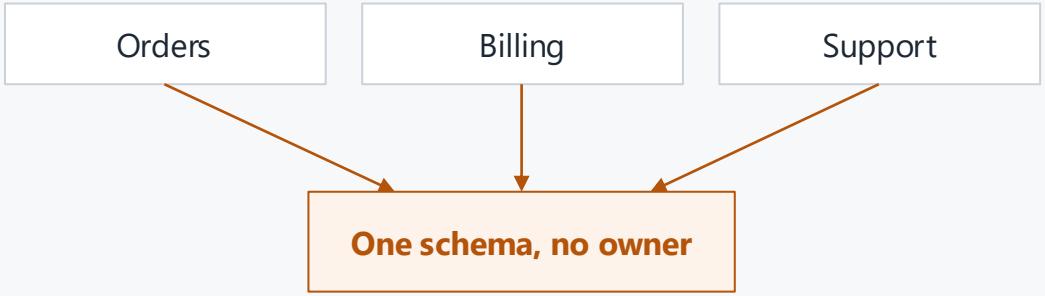
Size the aggregate to the invariant, never to the screen that displays it. One transaction commits one aggregate; the rest is reconciled through events.

The intuitive move is to load the whole object graph so the rule can be checked across it. That rebuilds a distributed transaction inside one process, serialises unrelated work, and makes the write path the slowest path in the system.

SESSION 05

Two instincts that re-couple the system you have just decoupled

ANTI-PATTERN 1 · THE SHARED DATABASE



WHY IT APPEALS

One schema, no duplication, joins still available and reporting is trivial.

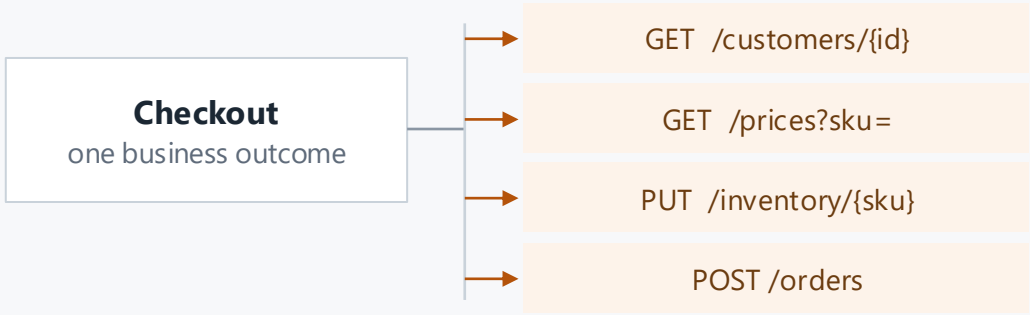
WHY IT FAILS

The schema becomes a public interface with no owner and no version. Every migration requires unanimous consent, so the database becomes the coupling the services were meant to remove.

INSTEAD

One owner per schema. Others read through a published contract, a read replica, or a projection built from events.

ANTI-PATTERN 2 · THE ENTITY SERVICE



WHY IT APPEALS

It resembles reuse: one service per noun, each owning its own table.

WHY IT FAILS

No service owns an outcome. One checkout becomes a chain of remote CRUD calls whose latencies add and whose failure modes multiply, and the invariant is enforced nowhere.

INSTEAD

Services own capabilities and outcomes rather than entities. Place the boundary where the business would name the process.

SESSION 06

Vivekanand Pujar | @VivekanandPujar

Six decisions every distributed design makes, deliberately or by accident

01 Partitioning

How is data and work divided?

Hash, range or tenant. The unit of division sets the unit of scale, and the unit of failure.

> *A single hot key becomes the ceiling of the entire system.*

02 Replication

How many copies, and do they agree?

Leader and follower, quorum, or consensus. Copies for availability are not copies for truth.

> *Read replicas quietly become the source of record for a report that matters.*

03 Coordination

What must be agreed, and by whom?

Locks, leases, consensus and idempotency keys. The most expensive primitive in the box.

> *Coordination adopted by default, and paid for on every request.*

04 Communication

A synchronous call, or an event?

Contracts, versioning, back-pressure and delivery semantics belong to this decision.

> *A synchronous chain that multiplies the unavailability of every hop.*

05 Resilience

What happens when a part is gone?

Timeouts, bulkheads, breakers, shedding and deliberate degradation.

> *No defined behaviour below full health, so the default is collapse.*

06 Observability

Can the behaviour be explained later?

Traces that cross boundaries, metrics that survive cardinality, and service objectives.

> *Incidents resolved by guesswork, and restarts recorded as fixes.*

The pillars are not layers. Each is a decision taken per capability, and the six answers together are what people mean by an architecture.

SESSION 07

A design the team cannot deploy safely is not yet a design



WHY THIS IS ARCHITECTURE RATHER THAN HYGIENE

Distribution multiplies environments. Every service requires somewhere to run, a network policy, a secret and a quota. Defined by hand, these diverge, and the divergence is the incident.

Independent deployability is the only return on the complexity already paid for. If releases must be coordinated, the result is a distributed monolith with none of the benefit.

Blast radius is a design parameter. Canary releases, feature flags and per-tenant rollout decide how much of the system a single bad change is able to reach.

THE OPERATING CONTRACT

Infrastructure as code — Terraform, OpenTofu, Pulumi, CloudFormation, Bicep or CDK, reviewed exactly as application code is reviewed.

Configuration and drift — GitOps through Argo CD or Flux; drift raises an alert rather than appearing in a quarterly audit.

Progressive delivery — Canary and blue-green releases, with automated reversal triggered by a breach of the objective.

Ownership — Error budgets, and the team that ships the service carries the pager.

The measure is the interval between a bad change reaching production and the system returning within its objective.

Vivekanand Pujar | @VivekanandPujar

CLOSING

Ten questions for the next design review

DATA

01 Which invariant genuinely requires linearizability, and which endpoint is paying for it?

02 Where does a write become visible, to whom, and after how long?

03 Which two systems are written in a single operation, and what makes that atomic?

BOUNDARIES

04 Does any feature in the backlog require two teams to release together?

05 Which aggregate enforces each rule, and is it sized to that rule or to a screen?

FAILURE

06 What is the behaviour when this dependency is slow rather than absent?

07 Who retries, how often, and against whose budget?

08 What is the defined degraded mode, and when was it last exercised?

OPERATIONS

09 Can this be deployed, observed and reversed without another team's involvement?

10 What is the blast radius of one bad change, and what bounds it?

Bring one service you own to the workshop. We will map its context boundaries, classify consistency per endpoint, and find the coordination point costing the most.

Recommended reading: Kleppmann, Designing Data-Intensive Applications; Evans, Domain-Driven Design.

Vivekanand Pujar | @VivekanandPujar